



THM

TECHNISCHE HOCHSCHULE MITTELHESSEN

**CAMPUS
GIESSEN**

MNI

Mathematik, Naturwissenschaften
und Informatik

**Entwicklung einer Wetter-App
Modul „Cloud Computing“(CS2025)
bei Stefen Rupp – Sommer Semester 2025**

Projektbericht

vorgelegt von

Dania, Al Aji
5460799

Eman, Kara Ali
5359518

Julienne Malvina, Tchenou Chimi
5363449

01. Juni 2025

Inhaltsverzeichnis

1	Einleitung	3
1.1	Projektüberblick.....	3
1.2	Fachliche Hintergrund.....	3
1.3	Aufgabenstellung und Anforderungen	3
1.4	Ausgewählte Technologien	4
2	Systemarchitektur.....	5
2.1	überblick über die beteiligten Systeme	5
2.2	Architekturdiagramm	5
2.3	Kommunikation und Datenfluss.....	6
3	Technische Umsetzung	7
3.1	Projektstruktur.....	7
3.2	Backend & Collector Services.....	7
3.3	Frontend und Visualisierung.....	7
3.4	Datenhaltung mit Pocketbase.....	7
4	CI/CD & Qualitätssicherung.....	8
4.1	GitLab CI/CD Pipeline	8
4.2	Linting & SonarQube	8
5	Nutzung & Bedienung	9
5.1	Nutzung der Weboberfläche.....	9
5.2	REST-API Übersicht.....	9
5.3	Beispielanwendung	10
6	Fazit und Ausblick	11
6.1	Zusammenfassung.....	11
6.2	Verbesserungsmöglichkeiten	11

1 Einleitung

1.1 Projektüberblick

Ziel dieses Projekts ist die Entwicklung einer Cloud-nativen Microservice-Anwendung zur Erfassung und Darstellung von Wetterdaten für verschiedene Geographische Standorte. Die Anwendung besteht aus mehreren entkoppelten Komponenten. Einem Webbasierten Frontend, einem Backend zur Steuerung und Verwaltung von Datensammelungsprozessen (sogenannte „Collector Services“) Sowie einer Backend-as-a-Service-Lösung (Supabase) zur Speicherung und Verwaltung der gesamten Daten. Der Nutzer kann Standorte hinzufügen, Wetterdaten abfragen lassen und diese grafisch oder tabellarisch einsehen.

1.2 Fachlicher Hintergrund und Herausforderung

Wetterdaten sind für viele Anwendungen von zentraler Bedeutung – von der Landwirtschaft über die Logistik bis hin zu Freizeit-Apps. Dabei ist es wichtig, Wetterinformationen kontinuierlich und automatisiert von externen Datenquellen (z. B. OpenWeatherMap) abzufragen und zentral zu speichern. Microservices bieten durch ihre Modularität eine skalierbare und wartbare Lösung, um diese Anforderungen effizient umzusetzen.

1.3 Aufgabenstellung und Anforderungen

Im Rahmen des Projekts sollten folgende Aufgaben umgesetzt werden:

- Entwicklung einer Weboberfläche zur Verwaltung von Standorten und zur Anzeige gesammelter Wetterdaten.
- Umsetzung eines skalierbaren Backends zur Verwaltung und Steuerung der Wetterdatensammlung.
- Implementierung von einzelnen Collector-Diensten, die in festgelegten Intervallen Wetterdaten abrufen.
- Speicherung aller Daten in einer Supabase-Datenbank über deren REST-API oder direkte Datenbankverbindung (PostgreSQL).
- Bereitstellung und Dokumentation von REST-Schnittstellen.
- Integration in GitLab inkl. CI/CD-Pipeline mit Linting und statischer Codeanalyse

1.4 Ausgewählte Technologien

Bereiche	Technologie
Frontend	Svelte
Backend	Express.js (Node.js)
Datenbank	Supabase(PostgreSQL BaaS)
Datenquelle	OpenWeatherMap API
CI/CD	GitLab CI/CD, SonarQube
Sontiges	REST, JSON, HTTP

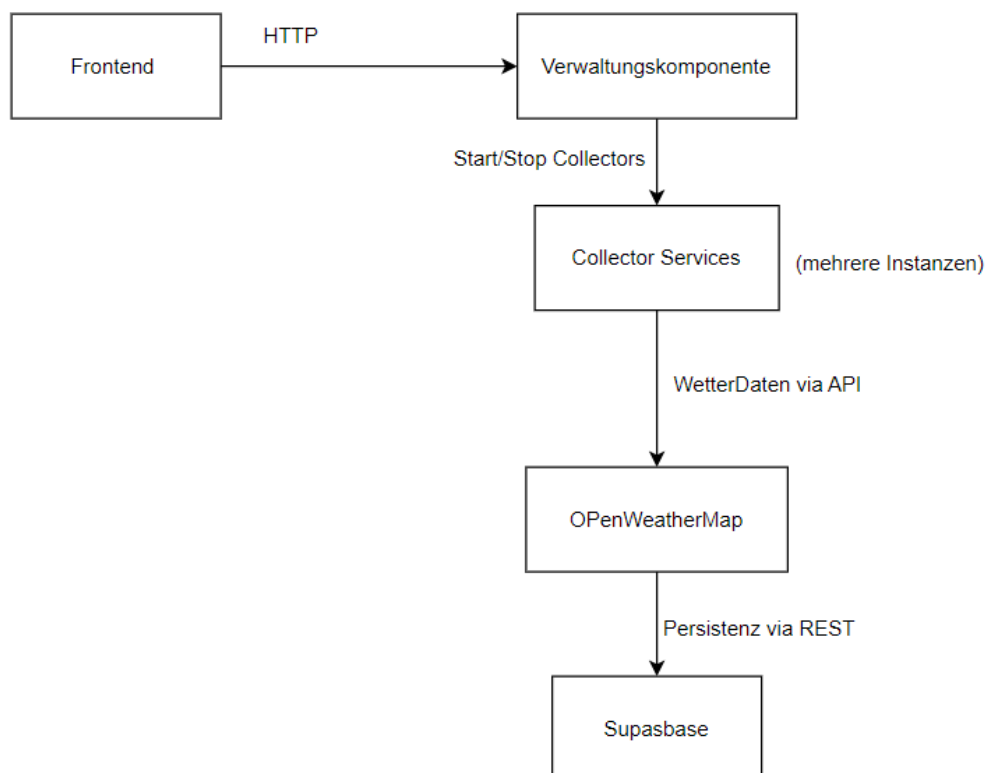
2 Systemarchitektur

2.1 Überblick über die beteiligten Systeme

Die Anwendung folgt einem Microservice-Ansatz und besteht aus mehreren voneinander getrennten Komponenten, die über HTTP-Schnittstellen miteinander kommunizieren. Dadurch wird eine hohe Skalierbarkeit und Wartbarkeit erreicht. Die wichtigsten beteiligten Systeme sind:

- **Frontend (Svelte Web-App):** Ermöglicht das Hinzufügen und Verwalten von Wetter-Standorten sowie die Visualisierung der gesammelten Wetterdaten.
- **Backend (Express.js):** Zentraler Steuerdienst, der die Collector-Services verwaltet. Stellt REST-APIs zur Verfügung, um Sammelprozesse zu starten, zu stoppen und zu überwachen.
- **Collector-Services:** Eigenständige Prozesse oder Dienste, die in konfigurierbaren Intervallen Wetterdaten (z. B. Temperatur, Luftfeuchtigkeit) über die OpenWeatherMap API abrufen.
- **Datenbank (Supabase):** Speichert die gesammelten Wetterdaten sowie Metadaten über Standorte. Supabase stellt hierfür sowohl ein PostgreSQL-Backend als auch REST- und Realtime-Schnittstellen zur Verfügung.
- **Drittanbieter-Wetterdienst (z. B. OpenWeatherMap):** Liefert aktuelle Wetterinformationen über eine öffentliche API.

2.2 Architekturdiagramm



2.3 Kommunikation und Datenfluss

Die Kommunikation und der Datenfluss in der Wetter-App verlaufen wie folgt:

- **Frontend → Backend (Express-API)**
Die Nutzerinteraktion erfolgt über das Svelte-Frontend. Wenn ein Nutzer einen neuen Standort zur Wetterüberwachung hinzufügen oder entfernen möchte, werden HTTP-POST-Anfragen an die Express-API des Node.js-Backends gesendet.
- **Backend → Collector (JavaScript-Objekt)**
Das Backend erzeugt für jeden Standort eine neue *Collector*-Instanz – dabei handelt es sich **nicht** um einen separaten Prozess oder Dienst, sondern um ein JavaScript-Objekt, das innerhalb der laufenden Node.js-Anwendung läuft. Die Collector-Instanz wird in einer Map gespeichert und führt in einem regelmäßigen Intervall HTTP-Abfragen an eine Wetter-API durch.
- **Collector → OpenWeatherMap API**
Die Collector-Instanz ruft regelmäßig aktuelle Wetterdaten von der OpenWeatherMap API für den konfigurierten Standort ab.
- **Collector → Supabase (über Supabase-JavaScript-Client)**
Die gesammelten Wetterdaten werden vom Collector an Supabase gesendet. Dazu wird der Supabase-JavaScript-Client verwendet, der die Daten als neue Einträge in die Datenbanktabelle `wetterdaten` einfügt.
- **Frontend → Backend/Supabase**
Für die Anzeige der Wetterdaten ruft das Frontend entweder direkt über Supabase (z. B. mit dem Supabase-Client in Svelte) oder indirekt über das Express-Backend (z. B. `/weather`) aktuelle und historische Wetterdaten ab. Optional kann dabei nach dem Ort gefiltert werden.

3 Technische Umsetzung

3.1 Projektstruktur

Das Projekt ist modular aufgebaut und besteht aus drei Hauptkomponenten:

- **Frontend:** Die Benutzeroberfläche wurde mit Svelte umgesetzt. Sie übernimmt die Verwaltung der Wetterdatenquellen, zeigt aktuelle Daten grafisch an und kommuniziert über REST mit dem Backend.
- **Backend:** Realisiert mit Express.js, stellt eine REST-API bereit zur Verwaltung von Collector-Services (Start/Stop/Status) und leitet die gesammelten Daten an Supabase weiter.
- **Datenbank & Authentifizierung:** Supabase übernimmt die Rolle des Backends-as-a-Service (BaaS) für Datenhaltung, bietet REST-Zugriff auf die gespeicherten Wetterdaten.

3.2 Backend und Collector Services

Das **Express.js-Backend** stellt eine REST-API bereit, über die neue Sammelprozesse gestartet oder beendet werden können. Es verwaltet alle Collector-Instanzen als separate Prozesse (z. B. über `child_process.spawn()` in Node.js).

Die **Collector-Services** sind so konzipiert, dass sie:

- in konfigurierbaren Intervallen (z. B. alle 10 Minuten) ausgeführt werden,
- Wetterdaten von einem bestimmten Ort über die OpenWeatherMap API abrufen,
- die Ergebnisse an Supabase senden.

3.3 Frontend und Visualisierung

Das **Svelte-Frontend** erlaubt es, neue Standorte hinzuzufügen, Sammelprozesse zu starten und die gesammelten Wetterdaten anzuzeigen – sowohl tabellarisch als auch grafisch (z. B. mit Chart.js oder ApexCharts).

- Kommunikation mit dem Backend erfolgt über REST-API.
- Datenvisualisierung erfolgt auf Basis von Werten aus Supabase.
- Die Anwendung ist responsiv und nutzerfreundlich gestaltet.

3.4 Datenhaltung mit Supabase

Supabase wird als zentrale Datenplattform genutzt und übernimmt folgende Funktionen:

- Speicherung von Wetterdaten in PostgreSQL-Tabellen.
- Authentifizierung und Autorisierung (optional).
- Zugriff über REST-API (Auto-generiert) und/oder SQL-Clients.

4 CI/CD und Qualitätssicherung

4.1 GitLab CI/CD Pipeline

Für eine kontinuierliche Integration und Bereitstellung wurde eine GitLab CI/CD Pipeline eingerichtet. Diese automatisiert zentrale Aufgaben wie Code-Analyse, Build und Tests.

- Aufbau der Pipeline

Die CI/CD Pipeline ist in mehrere Jobs unterteilt, definiert in der Datei `.gitlab-ci.yml`. Die wichtigsten Jobs sind:

- **Install:** Installiert alle Abhängigkeiten für Backend und Frontend.
- **Lint:** Führt statische Codeanalysen durch (z. B. ESLint für JS/TS).
- **Build:** Erstellt den Produktions-Build für das Frontend.
- **Deploy:** Stellt den Code auf einem Server oder Container bereit

Die Pipeline wird bei jedem Push automatisch ausgeführt. Fehlerhafte Commits werden durch die Jobs direkt blockiert, wodurch die Codequalität im Hauptzweig gewahrt bleibt.

4.2 Linting und SonarQube

Zur Verbesserung der Codequalität und Wartbarkeit wurden Linting-Tools und statische Codeanalyse mit SonarQube eingesetzt.

- **Linting**
 - ESLint wurde für JavaScript und Svelte verwendet.
 - Standardregeln wurden ergänzt um empfohlene Regeln zur Codekonvention, wie z. B. semantische Benennung, keine unbenutzten Variablen, usw.
- **SonarQube**

SonarQube wird als statisches Analysewerkzeug in der GitLab-Pipeline verwendet. Es überprüft:

- Code Smells (unsauberer oder ineffizienter Code)
- Bugs und potenzielle Laufzeitfehler
- Sicherheitsprobleme
- Testabdeckung (wenn Tests vorhanden sind)

5 Nutzung und Bedienung

5.1 Nutzung der Weboberfläche

Die Webanwendung (Svelte) bietet eine einfache Benutzeroberfläche zur Verwaltung und Anzeige von Wetterdaten.

Hauptfunktionen:

- Standort hinzufügen: Benutzer geben den Namen einer Stadt ein und starten einen Datensammler für diese Region.
- Datenübersicht: Gesammelte Wetterdaten wie Temperatur, Luftfeuchtigkeit oder Luftdruck werden grafisch (z. B. mit Diagrammen) und tabellarisch angezeigt.
- Live-Aktualisierung: Die Daten werden regelmäßig aktualisiert (z. B. alle 10 Minuten), ohne dass die Seite neu geladen werden muss.
- Statusanzeige: Zeigt aktive Collector-Dienste an, inkl. Möglichkeit zum Stoppen oder Neustarten.

5.2 REST-API Übersicht

Das Backend stellt eine REST-API bereit, über die Collector-Services gesteuert werden können.

Wichtige Endpunkte:

Methode	Pfad	Beschreibung
POST	/collectors/start	Startet einen Collector für einen Standort
POST	/collectors/stop	Stoppt einen Collector
GET	/collectors/status	Gibt Status aller laufenden Dienste zurück
GET	/weather?city=Berlin	Ruft Wetterdaten für einen Ort ab

5.3 Beispielanwendung

Ein typischer Ablauf in der Anwendung:

1. **Nutzer öffnet die App** im Browser.
2. **Klick auf „Neuen Standort hinzufügen“**, z. B. „*Berlin*“.
3. Die App sendet einen Request an das Backend → Collector wird gestartet.
4. Der Collector ruft alle 10 Minuten Wetterdaten über OpenWeatherMap ab.
5. Die gesammelten Daten werden in Supabase gespeichert.
6. Die Web-App ruft die aktuellen Daten ab und zeigt Temperaturverläufe über die Zeit in einem Liniendiagramm an.

6 Fazit und Ausblick

6.1 Zusammenfassung

Im Rahmen dieses Projekts wurde eine cloud-native, Microservice-basierte WetterApp erfolgreich umgesetzt. Die Anwendung besteht aus einer Svelte-Weboberfläche, einem Express.js-Backend zur Verwaltung von Collector-Services sowie individuell konfigurierbaren Wetter-Diensten, die Daten regelmäßig über die OpenWeatherMap-API erfassen.

Die gesammelten Wetterdaten werden zentral in Supabase gespeichert und über die Web-App übersichtlich dargestellt. Durch die modulare Architektur ist die Anwendung gut skalierbar und erweiterbar – neue Collector-Dienste oder Visualisierungen können ohne großen Aufwand integriert werden.

Zusätzlich wurden moderne Entwicklungs- und Qualitätssicherungsprozesse wie GitLab CI/CD, ESLint und SonarQube eingesetzt, um eine stabile und wartbare Codebasis sicherzustellen.

6.2 Verbesserungsmöglichkeiten

Trotz der funktionalen Umsetzung bestehen mehrere sinnvolle Erweiterungspotenziale:

- **Erweiterung der Datenquellen:** Neben OpenWeatherMap könnten weitere APIs (z. B. Wetterdienstleister mit genaueren Vorhersagen oder Luftqualitätsdaten) eingebunden werden.
- **Erweiterte Benutzerverwaltung:** Authentifizierung über Supabase, um benutzerspezifische Datenquellen und Einstellungen zu ermöglichen.
- **Alert-System:** Benachrichtigungen bei extremen Wetterbedingungen per E-Mail oder Push-Message.
- **Skalierung mit Containern:** Der Einsatz von Docker/Kubernetes könnte die Collector-Services effizienter verwalten lassen.
- **Testabdeckung:** Automatisierte Tests für Backend-Logik und Frontend-Komponenten könnten weiter ausgebaut werden.